Informe de módulo — M1 Cuentas, roles y seguridad

Plataforma de Simulaciones Académicas · Universidad de La Guajira

1. Identificación

- **Módulo:** M1 Cuentas, roles y seguridad
- **Responsable:** Diego Andrés Maldonado Urueta
- **Fecha:** 26 de octubre de 2026 · **Versión del informe:** 0.1 (diseño + migración SQL, sin probar)
- **Destinatarios:** todo el equipo y los asistentes de IA de cada integrante, en especial M6 (Luis Cotes, Samir Iguarán), M4 (Jhonner Bernal) y M5 (Jorge Córdoba).
- **Archivos entregados:** `supabase/migrations/0001_m1_perfiles_y_roles.sql` y su reversa `supabase/migrations_down/0001_m1_perfiles_y_roles.down.sql`.

Estado real, sin adornos: el diseño está completo y el SQL está escrito, pero **no se ha ejecutado contra PostgreSQL ni Supabase** (en el entorno donde se redactó no había PostgreSQL disponible). Hay que correrlo y probarlo antes de darlo por bueno. No hay interfaz de M1 todavía.

2. Requerimientos cubiertos

Código	Estado
RF-01 Registro e inicio de sesión con correo	Diseñado sobre Supabase Auth; el trigger de alta de perfil está escrito. Pantallas pendientes.
RF-02 Roles y permisos	Modelo de 4 roles con sesión + visitante, funciones de permiso y RLS escritos.
RF-03 Perfil con carrera y semestre	Tabla `perfiles` con `carrera_id` y `semestre`. La FK a carreras la agrega M2.
RF-04 Recuperación de contraseña	Se delega a Supabase Auth (`resetPasswordForEmail`); no requiere tablas propias. Pantalla pendiente.
RF-19 Aislamiento de contenido de usuarios	**Pendiente.** Es de ejecución en el navegador (iframe sandbox / Worker) y se diseña junto con M4 y M5. No está en esta migración.

Código	Estado
RNF-02 Seguridad por rol	RLS activo en `perfiles` y `cambios_rol`.

3. Qué se hizo y por qué

3.1 Modelo de roles

- **visitante:** no tiene cuenta; en Supabase es el rol de base de datos `anon`. No hay fila en `perfiles`.
- **estudiante, profesor, moderador, administrador:** columna `perfiles.rol` con `CHECK`.
- **Decisión:** un usuario tiene **un solo rol**. Un administrador cuenta también como moderador (`es_moderador()` es verdadero para ambos) y como profesor (`es_profesor()`).
- **Razón:** es lo más simple que cumple la tabla de actores del documento general. Si luego se necesita que alguien sea profesor y moderador a la vez, habría que pasar a una tabla de roles múltiples; queda como pendiente.

3.2 Alta de cuenta y perfil

- El registro y el login los maneja **Supabase Auth** (`auth.users`); no se guarda ninguna contraseña en tablas propias.
- Un trigger `trg_crear_perfil` crea la fila de `perfiles` al registrarse, con el rol **siempre en `estudiante`**.
- **Decisión de seguridad:** el rol nunca se lee de los metadatos enviados por el cliente, porque cualquiera podría registrarse enviando `rol: administrador`.
- El nombre se toma de `raw_user_meta_data->>'nombre_completo'` (recortado a 120 caracteres) y es el único dato que viene del cliente.

3.3 Cómo se asignan los roles

- La única vía para cambiar un rol es la función `asignar_rol(p_usuario, p_rol, p_motivo)`.
- Solo la puede ejecutar un administrador, no puede cambiarse el rol a sí mismo y cada cambio queda en la tabla `cambios_rol` (quién, a quién, antes, después, motivo, fecha). Esto da trazabilidad.
- Un usuario normal **no puede** actualizar su columna `rol`: el `GRANT UPDATE` solo cubre `nombre_completo`, `carrera_id` y `semestre`.

- **Primer administrador:** se asigna a mano con un `UPDATE` una sola vez (instrucción al final del SQL). No va dentro de la migración para no dejar un administrador por defecto en el repositorio.
- **Profesores:** hoy los promueve un administrador. **Pendiente de decisión del grupo:** si se asigna automáticamente por dominio de correo institucional. No conozco el dominio exacto ni la política de la universidad, así que no lo asumí.

3.4 Funciones de permiso

Todas son `SECURITY DEFINER` con `search_path` fijo, para evitar recursión de RLS y secuestro de funciones.

- `rol_actual() → text`
- `es_administrador() → boolean`
- `es_moderador() → boolean` (moderador **o** administrador) — **contrato con M6**
- `es_profesor() → boolean` (profesor **o** administrador)

3.5 Seguridad a nivel de fila (RLS)

- `perfiles`: cada usuario lee solo su fila; el administrador lee todas. La actualización está limitada a la fila propia y a tres columnas. No se otorgan `INSERT` ni `DELETE` a clientes (el borrado ocurre en cascada desde `auth.users`).
- `cambios_rol`: solo lectura para administradores; nadie escribe directo.
- `anon` no tiene acceso a ninguna tabla de M1.

3.6 Vista pública `perfiles_publicos`

- Respuesta al pendiente de M6 sobre mostrar el nombre del autor en comentarios.
- Expone **solo** **`id`** **y** **`nombre_completo`**; nunca correo, rol, carrera ni semestre.
- **Punto a revisar:** la vista corre con los permisos de su dueño, lo cual es intencional para saltar la RLS de `perfiles`. Hay que verificar en Supabase que se comporta así y valorar si conviene limitarla a `authenticated`.

4. Interfaces con otros módulos

M1 ofrece:

- Tabla `public.perfiles(id uuid, nombre_completo, correo, rol, carrera_id, semestre, creado_en, actualizado_en)`, con `id` igual a `auth.users.id`.
- Funciones `es_moderador()`, `es_administrador()`, `es_profesor()`, `rol_actual()`.
- Función `asignar_rol(uuid, text, text)` (solo administrador).

- Vista `perfiles_publicos(id, nombre_completo)`.
- Identidad: `auth.uid()` es el usuario actual; los IDs son `uuid`.

Confirmaciones a M6:

- `perfiles(id uuid)` ligada a `auth.users` queda confirmada.
- `es_moderador() returns boolean` queda confirmada, con la salvedad de que **incluye administradores**.
- Roles múltiples: no, un rol por usuario.

M1 espera de los demás:

- **M2:** tabla `public.carreras(id uuid)`. M2 agrega la FK `perfiles.carrera_id → carreras(id)` en su propia migración.
- **M4/M5:** usar `es_profesor()` para el distintivo oficial (RF-21) y `es_moderador()` para moderación (RF-18); no leer `perfiles.rol` directamente desde sus políticas.
- **Interfaz (M6):** nunca usar la clave `service_role` en el navegador; solo la clave pública (`anon`) y la sesión del usuario.

5. Modelo de datos afectado

- **Nuevas:** `perfiles`, `cambios_rol`, vista `perfiles_publicos`.
- **Nuevos objetos:** trigger `trg_crear_perfil` sobre `auth.users`, trigger `trg_perfiles_actualizado`, y las funciones listadas en 3.4 y 3.3.
- **Orden de aplicación:** M1 0001 va **antes** que M6 0001–0004. Para revertir, el orden inverso.
- Los stubs de M6 (`tests/00_stubs_provisionales.sql`) deben sustituirse por esta migración. El stub de `perfiles` solo tiene `id`; la real tiene `correo` obligatorio, así que los inserts de prueba de M6 deberán pasar por `auth.users` (que dispara el trigger) o completar `correo`.

6. Cómo probarlo

No está hecho todavía. Plan propuesto:

1. Aplicar la migración en un Supabase de desarrollo (o en PostgreSQL local con los stubs de `auth`).
2. Crear dos usuarios y comprobar que cada uno tiene su perfil con rol `estudiante`.
3. Comprobar que un estudiante **no** puede hacer `update perfiles set rol = 'administrador'` sobre sí mismo.

4. Comprobar que `asignar_rol` falla para no administradores y para el propio administrador, y que escribe en `cambios_rol`.
5. Comprobar que un estudiante no lee el perfil de otro y que `perfiles_publicos` sí devuelve solo `id` y nombre.
6. Reaplicar las migraciones de M6 sobre las tablas reales y correr `tests/run_tests.sh`.
7. Aplicar la reversa y volver a aplicar.

7. Problemas conocidos y pendientes

- SQL **sin ejecutar**: puede tener errores de sintaxis o de permisos.
- RF-19 (aislamiento de contenido) sin diseñar; depende de M4/M5.
- Pantallas de registro, login, recuperación y perfil sin hacer (la interfaz está aplazada por acuerdo del equipo).
- Política de correo institucional y de asignación de profesores sin decidir.
- Un solo rol por usuario; no hay desactivación o bloqueo de cuentas (por ejemplo, un campo `activo`).
- Sin límite de intentos ni verificación de correo configurados: se hacen en los ajustes de Supabase Auth y deben activarse.
- Pruebas automáticas de M1 sin escribir.

8. Cambios solicitados a otros módulos

- **M2:** crear `carreras(id uuid)` y la FK desde `perfiles.carrera_id`.
- **M4/M5:** usar las funciones de permiso de M1 en lugar de leer `perfiles.rol`, y confirmar que `simulaciones.autor_id` referencia `perfiles(id)`.
- **M6:** reemplazar los stubs por esta migración y reprobar; usar `perfiles_publicos` para el nombre del autor en comentarios.
- **Líder:** decidir la política de profesores y de correo institucional.

9. Contexto para el siguiente integrante

M1 es la base de identidad: todo usa `auth.uid()` y los `uuid` de `perfiles`. Hay cuatro roles con sesión más el visitante (`anon`), un rol por usuario. El rol nace siempre como estudiante y solo un administrador lo cambia con `asignar_rol`, con registro de auditoría. Para permisos en políticas RLS use `es_moderador()`, `es_profesor()` y `es_administrador()`, no lea la columna `rol`. Para mostrar nombres use `perfiles_publicos`. El SQL **no está probado**: lo primero es ejecutarlo.